A HYBRID APT DETECTION AND INVESTIGATION SYSTEM

Complete Project Guide: GAT + GRU + VGAE

DARPA Transparent Computing • Provenance graphs • Temporal learning • Anomaly scoring • Attack-chain reconstruction

Research-grounded design guide | Version 1.0 | 25 October 2026

Based on the three uploaded papers: MGDA, Commander, and EdgeFence

How to use this guide

This document is a build plan for an experimental APT detector. It moves from dataset selection through parsing, provenance-graph construction, chronological splitting, model design, training, scoring, evaluation, attack investigation, and dashboard output. The diagrams are original renderings created for this project so the labels remain readable when printed.

Important architecture distinction

The three uploaded papers do not present one combined GAT–GRU–VGAE model. MGDA uses a multi-view masked graph autoencoder with a GAT encoder plus KNN-based outlier scoring and TTP-based attack reconstruction; Commander is a provenance-tracing and rule-based multi-stage detector; EdgeFence uses temporal graph snapshots, a lightweight GraphSAGE-style GNN and GRU with federated learning. This guide combines selected methods into a proposed hybrid and labels those adaptations explicitly.

Contents

- 1. Executive summary and recommended project scope
- 2. What the three uploaded papers contribute
- 3. Research objectives and measurable questions
- 4. Dataset selection: DARPA Transparent Computing
- 5. Development environment and resource plan
- 6. Project phases: raw files to graph dataset
- 7. Labels, ground truth and leakage-safe splitting
- 8. GAT: internal layers, equations and use
- 9. GRU: internal gates, equations and use
- 10. VGAE: latent space, decoder and anomaly scoring
- 11. Proposed hybrid training objective
- 12. Inference, anomaly score and thresholding
- 13. Attack scenario reconstruction and alert output
- 14. Evaluation plan and ablation experiments
- 15. API/dashboard and repository layout
- 16. Implementation roadmap and acceptance criteria
- 17. Common failure modes and mitigations
- 18. Final deliverables / viva explanation
- 19. References and paper-to-design traceability

1. Executive summary and recommended project scope

Build a provenance-graph detector that learns relationships between system entities, behavior across time windows, and the likelihood of observed graph edges. GAT provides spatial/relational representations; GRU models temporal evolution; VGAE supplies probabilistic latent representations and graph-link reconstruction. A separate detector turns model residuals into calibrated anomaly scores. A causal-tracing and TTP-rule module transforms suspicious seeds into a small investigation graph and a structured alert.

Recommended first version (MVP)

1. Use one known-good DARPA TC Engagement 3 (E3) performer/run, preferably CADETS or THEIA, rather than trying to ingest every performer and engagement at once. The MGDA paper evaluates CADETS,

THEIA and TRACE E3; the official E3 README lists validated data topics and the E3 ground-truth report [1,4].

2. Convert records to a normalized event table with timestamp, source/subject, operation/relation, destination/object, host and event identifier.
3. Build directed, typed provenance graph snapshots using time windows. Keep labels and ground-truth indicators out of neural-network features.
4. Train an edge-aware GAT per snapshot; pass per-entity embeddings across past snapshots to GRU; use fused spatial/temporal context for a VGAE-style latent edge predictor.
5. Train first on benign windows; calculate next-window edge surprise and a KNN distance over benign embeddings. Calibrate an alert threshold on validation data.
6. For each alert, trace only a bounded causal neighborhood and use documented TTP rules for attack-stage interpretation. Store JSON evidence and render a small attack graph.
7. Evaluate the full hybrid against simpler ablations and report precision, recall, F1, false-positive rate, PR-AUC and detection delay.

What this project does and does not claim

- Does: detect graph/time-window behavior that is poorly explained by a trained benign model, rank suspicious entities or windows, and help analysts examine related activity.
- Does: adapt methods from the uploaded papers and test whether their combination improves the measured result.
- Does not: guarantee that an anomaly is an APT, automatically infer a complete MITRE ATT&CK technique from a score alone, or reproduce the performance reported in the papers without using the same data, splits and experimental environment.
- Does not: make federated learning, CKKS encryption, or adversarial training part of the MVP. These are optional later extensions.

2. What the three uploaded papers contribute

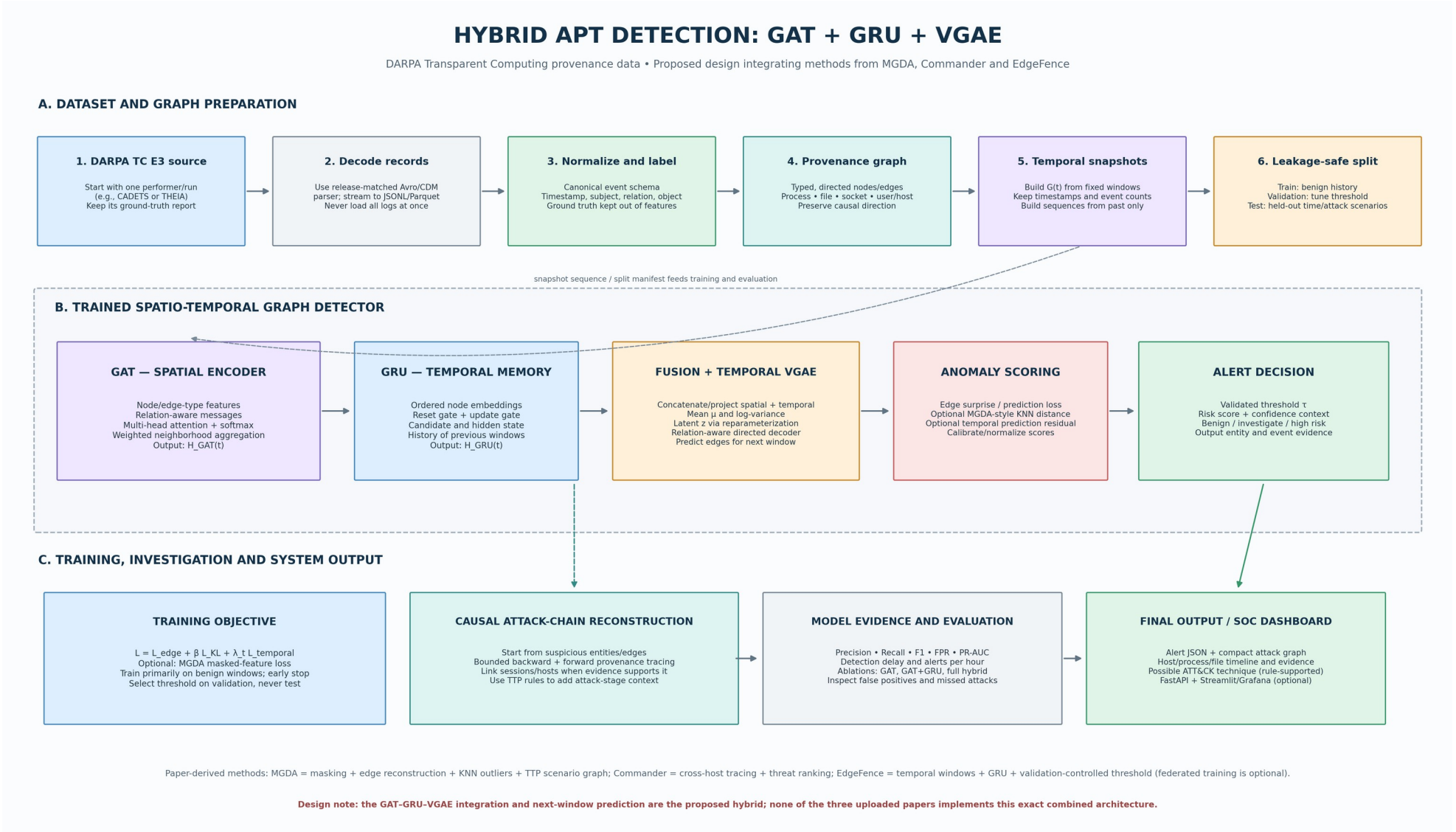
Uploaded paper	Method described in that paper	How to use it in this project	Important boundary
MGDA: A provenance graph-based framework for threat detection and attack scenario reconstruction (Cui et al., 2026) [1]	Directed provenance graph construction; semantics-preserving compression; one-hot node/edge types; stacked GAT encoder; masked node feature reconstruction with multi-view re-masking and latent representation prediction; sampled edge reconstruction; KNN outlier score over benign embeddings; TTP-rule attack-scenario reconstruction.	Use its graph schema, efficient event handling, relation-aware GAT ideas, self-supervised pretraining, KNN anomaly score, and TTP/attack-graph investigation approach.	MGDA is not a GRU+VGAE system. Its decoder/objectives and KNN detector are specific to MGDA; borrow them as auxiliary components or baselines, not as proof of the hybrid. Paper tests include CADETS/THEIA/TRACE E3, StreamSpot and Unicorn Wget.

Uploaded paper	Method described in that paper	How to use it in this project	Important boundary
Commander: A robust cross-machine multi-phase APT detector via provenance analytics (Liu et al., 2025) [2]	Three-stage system: parallel preliminary detectors (authentication anomaly, persistence, session hijacking, port forwarding); logon-session-aware backward/forward tracing and cross-machine tracing; TTP rule matching and threat-score ranking.	Use the investigation pattern: suspicious seeds -> bounded backward/forward tracing -> correlate sessions/hosts when evidence supports it -> TTP-rule context -> rank compact attack graphs.	Commander is a separate provenance analytics/rule system, evaluated on AVIATOR and industrial/Active Directory scenarios. Its specialized Windows/ICS rules are not automatically compatible with DARPA TC CDM records.
EdgeFence: Federated Temporal Graph Neural Networks for Lightweight, Adversarial Malware Detection in Distributed Edge Networks (Isaac et al., 2026) [3]	Fixed-interval graph snapshots, temporal windows, a lightweight three-layer GraphSAGE-inspired GNN with mean aggregation, global mean pooling, one-layer GRU (64 hidden units in the described setup), sigmoid maliciousness score, validation threshold by target FPR, plus optional FedAvg and adversarial training.	Use snapshot-sequence construction, GRU temporal modelling, threshold tuning on a validation set, causal label assignment to the latest snapshot, and latency profiling.	EdgeFence uses a GraphSAGE-style spatial encoder and reports experiments on CIC-IDS2018 and IoT-23, not DARPA provenance graphs. Use its temporal method as design guidance; do not transfer its metrics or hyperparameters uncritically.

2.1 Original algorithm foundations

- GAT: learnable attention weights over graph neighborhoods; use relation-aware messages for the typed provenance graph [5].
- GRU: reset and update gates retain/use sequence history while generating the next hidden state [6].
- VGAE: variational node embeddings, latent Gaussian parameters and graph decoder/reconstruction objective [7]. The original VGAE uses an inner-product decoder; directed, typed next-window prediction is an explicit adaptation in this guide.
- DARPA Transparent Computing: high-fidelity provenance and causal dependencies are intended to “connect the dots” between individually benign-looking activities that collectively indicate malice [4].

Figure 1. End-to-end project architecture



Original diagram. Solid path = data/model flow. The GAT–GRU–VGAE combination and causal next-window prediction are the proposed hybrid; the bottom notes identify methods adapted from each source paper.

3. Research objectives and measurable questions

Project objective

Design and evaluate a hybrid provenance-graph detector that uses GAT to learn spatial dependencies, GRU to learn temporal behavior, and a VGAE-style decoder to estimate how surprising observed directed, typed edges are given prior graph history. The detector should generate evidence-backed investigation alerts and compact attack scenario graphs.

Research questions

1. Does GAT learn useful structural context from provenance graphs compared with shallow graph statistics or feature-only baselines?
2. Does GRU over ordered graph snapshots improve detection of multi-step behavior compared with a single-snapshot GAT?
3. Does the VGAE-style edge-prediction residual improve detection over embedding-distance/KNN scoring alone?
4. Does causal tracing plus TTP-rule matching make alert evidence more compact and interpretable than a list of anomalous nodes?
5. What are the trade-offs among detection quality, false-positive rate, inference latency, memory use and graph size?

Proposed hypotheses

- H1: A GAT representation will retain contextual information that degree/rule-only features miss.
- H2: Ordered GRU history will identify some multi-event patterns that are ambiguous in one snapshot.
- H3: Combining edge-surprise with KNN embedding distance will provide complementary evidence, but the improvement must be demonstrated through ablation.
- H4: A bounded tracing and rule layer will make analyst output more interpretable without claiming that every detected anomaly is malicious.

4. Dataset selection: DARPA Transparent Computing

4.1 Start with a specific engagement and performer

Do not treat “DARPA TC” as one uniform table. TC releases include different engagements and performer outputs (e.g., CADETS, THEIA, TRACE, CLEARSCOPE and Five Directions), with differences in host platform, event coverage and available records. The official E3 release provides a machine-readable CDM Avro schema, a human-readable CDM description, a Java consumer that can convert Avro binary to JSON, an operational event log, and an E3 ground-truth report [4].

Choice	Recommendation	Reason / caveat
Recommended MVP	DARPA TC E3; one known-good CADETS or THEIA run.	MGDA evaluates E3 CADETS/THEIA/TRACE and the official README lists known-good topics. Start with one run to validate parsing and labels before scaling.

Choice	Recommendation	Reason / caveat
If the file is from E5	Use E5 schema, parser instructions, and ground truth; do not parse as E3.	Engagement 5 uses a different release package. A filename ending in `e5` is not interchangeable with E3. Large E5 collections can expand substantially after extraction or loading.
Later generalization test	Add another E3 performer or a separate run only after the first run works.	This helps test transfer across logging producers but must be reported as a separate experiment because event semantics and coverage may differ.

4.2 Official dataset starting points

- DARPA TC program overview: https://www.darpa.mil/research/programs/transparent-computing [4].
- Official E3 release README and file manifest: https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md [4].
- Official repository: https://github.com/darpa-i2o/Transparent-Computing. Read the README for the selected E3/E5 release and ground-truth report before converting files.
- For E3, the README says the release includes `ta3-java-consumer.tar.gz`, the CDM Avro schema, and a script to convert Avro binary to JSON; its consumer requires Java 8 and building from source requires Maven [4].

Resource warning

DARPA TC provenance data is large and may expand significantly after decoding or loading into a database. Start with one known-good topic, process one file/chunk at a time, save normalized Parquet/JSONL outputs, and profile graph size. Do not begin by loading all E3/E5 data into RAM.

4.3 Dataset inspection checklist

1. Write down the engagement (E3 or E5), performer (CADETS/THEIA/etc.), exact topic/run, original filename(s), compressed size, checksum and data source.
2. Keep the downloaded archive immutable. Store parser output separately from raw files.
3. Read the matching ground-truth report and operational-event log. Identify documented attack windows, attack storylines, IOCs and known data quality issues.
4. Run the release-matched decoder on a small file first. Confirm record counts, record types, timestamps and entity references.
5. Check missing IDs, malformed records, duplicate event IDs, timestamp ranges, host coverage and event counts per interval.
6. Only after the parser works, process the chosen run incrementally and write chunked JSONL/Parquet. Create a manifest of every input file and output partition.

5. Development environment and resource plan

Recommended implementation stack: Python, PyTorch, PyTorch Geometric, pandas or Polars, PyArrow, scikit-learn, NetworkX for small graph diagnostics, FastAPI for inference endpoints, and Streamlit or Grafana for a demonstration dashboard. Use the official parser for the chosen CDM release rather than assuming that a generic JSON parser can read binary Avro records [4,8].

5.1 CPU-first environment commands (Windows PowerShell)

```
# From the project root; use Python 3.11 if it is available and compatible.
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip

# CPU-only PyTorch build; then install PyTorch Geometric.
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install torch-geometric pandas pyarrow numpy scikit-learn networkx
pip install matplotlib seaborn fastapi uvicorn streamlit pydantic joblib

# Verify the environment
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
python -c "import torch_geometric; print(torch_geometric.__version__)"
```

Package versions can change. If installation fails, use the official PyTorch and PyTorch Geometric installation instructions and choose mutually compatible versions [8]. For DARPA E3 Avro conversion, install/use the Java version required by the official consumer. Keep parser tooling separate from the Python model environment.

5.2 Practical starting configuration

Setting	Start here	Why / tuning rule
Data volume	One performer/run; first inspect a limited time interval and a few thousand nodes.	Validate correctness before attempting the full trace.
Graph window	Start with 60 seconds; compare 30 seconds, 1 minute and 5 minutes on validation.	Time-window size changes graph density and detection delay; do not copy EdgeFence's 1-second interval without measurement.
GAT	2 layers; 32–64 hidden dimensions; 2–4 attention heads; dropout around 0.2.	Compact CPU baseline; use edge/relation embeddings if implemented.
GRU	1 layer; 32–64 hidden units; history length 5–10 snapshots.	EdgeFence uses a one-layer GRU with 64 hidden units; treat this as a reference, not a required setting [3].
VGAE latent dimension	16–32 per node for the first CPU experiment.	Use a directed/relation-aware decoder for typed provenance edges.
Optimizer	Adam; learning rate 0.001; weight decay 0 or 0.0005 as a starting grid.	MGDA reports Adam, learning rate 0.001 and a weight-decay setting around 5e-4; tune on validation [1].
Batch size	One small graph window sequence; increase only after profiling.	Avoid memory spikes from large adjacency matrices.
Training duration	Start with 20–50 epochs plus early stopping.	Select based on validation loss, not a fixed epoch count.
KNN anomaly score	k=10 as a first baseline.	MGDA uses KNN with a K-D tree and normalizes local distance; test other k values if the embedding size permits [1].

Hardware fit

For a CPU-only computer with about 8 GB RAM, a full DARPA TC E5 pipeline may be impractical to decode and train in one pass. Use streaming conversion, compact time windows, subgraphs/neighbor sampling, and save intermediate Parquet files. Increase scale only after end-to-end correctness is proven.

6. Project phases: raw files to graph dataset

Phase 0 — Scope, repository and reproducibility

- Fix the initial dataset release and performer; record the source README and ground-truth version.
- Define detection unit: entity, event/edge, window, or incident. A workable MVP scores graph windows plus important entities.
- Create a configuration file for parser paths, window size, node/edge types, seed, split boundaries and model settings.
- Use a fixed random seed for model training and random negative-edge sampling; log Python/PyTorch/PyG versions.
- Keep raw dataset files out of Git; commit the parser, mappings, configs, small synthetic tests and documentation.

Phase 1 — Decode and create a normalized event table

Use the release-specific DARPA consumer to decode Avro/CDM records. Map relevant source records to a canonical schema. Do not assume all rows are the same CDM record type; object records and event records may need to be joined using UUIDs or the release-defined identifiers.

Canonical field	Meaning	Required handling
event_id	Stable identifier for an event if available	Preserve the original ID for trace-back.
timestamp	Event time	Normalize to one timezone/unit; keep raw timestamp too.
host_id	Host or agent identifier	Preserve performer/run context; never merge identical IDs across hosts unless schema says they are global.
subject_id / subject_type	Initiating entity, e.g. process	Resolve entity records and process lifetime where possible.
relation_type	Action: READ, WRITE, EXECUTE, CONNECT, FORK, etc.	Map the source vocabulary to a stable controlled vocabulary; retain original relation.
object_id / object_type	Target entity, e.g. file or socket	Resolve object records and preserve identifiers.
edge_direction	Subject -> object direction	Do not convert a directed causal event into an undirected relation by accident.
attributes	Command line/path/UID/IP/port/etc.	Store for later interpretation; decide explicitly which are model features.
ground_truth_tag	Label from report/IOC mapping	Keep in a separate label table; never include in the model feature vector.

Phase 2 — Data quality and semantic compression

- Check invalid timestamps, missing subject/object IDs, unknown relation types and duplicate events.
- Preserve `event_id`, raw fields, parser version and source file/offset where possible so every alert can be traced back.
- MGDA removes only semantically equivalent redundant events where doing so preserves data dependencies and meaning [1]. For this hybrid, preserve occurrence counts and timing gaps if they may be useful to GRU.
- Do not silently merge processes with the same name; include host, process identifier and lifetime context.
- Write clean event chunks to Parquet and produce data-quality summaries before graph construction.

Phase 3 — Construct provenance graphs

Represent each graph as G_t = (V_t, E_t, X_t), with node and edge metadata. A node may be a process, file, network socket, user or host. An edge records an observed interaction and is directed from subject to object. For example, PROCESS_1 -> READ -> FILE_9. If the selected release does not expose a user/host as a direct event participant, do not invent those edges; use only relationships supported by the record schema.

Graph element	MVP representation	Advanced extension
Node type	One-hot or learned embedding for process/file/socket/other supported types.	Semantic features such as executable path, file class, privilege, host role, properly encoded and leakage-tested.
Edge type	One-hot or learned relation embedding.	Edge-aware GAT message passing; relation-specific decoder for directed typed links.
Time	Timestamp and optional time-delta bucket; graph window index.	Time decay, inter-event intervals, per-entity event frequency.
Identity	Stable entity ID plus host/run namespace.	Entity lifetime tracking, process ancestry and session IDs when supported.
Graph size control	Filter invalid records and use subgraphs; document filtering rules.	Semantics-preserving duplicate compression and sampling with trace-back.

Phase 4 — Create temporal windows

```
for each time window t in chronological order:
    events_t = events with timestamp in [t, t + delta)
    nodes_t, edges_t = build_provenance_graph(events_t)
    X_t = encode_node_features(nodes_t)
    A_t = encode_edges(edges_t)  # preserve direction and relation type
    save_snapshot(t, X_t, A_t, node_ids, event_ids)

# At detection time, use only past snapshots to predict the current one.
sequence_t = [G_(t-w), ..., G_(t-1)]
target_graph = G_t
```

The notation uses a fixed interval Δ and a history length w. The exact interval should be selected empirically. EdgeFence describes fixed-interval graph snapshots and a sequence of the most recent w snapshots, with the label corresponding to the latest/current snapshot to keep the task causal [3].

Phase 5 — Prepare labels without contaminating features

Build a separate ground-truth mapping table from the selected report and available records. Link known IOCs and documented attack activity to event IDs, time intervals, host IDs, file/process attributes or edges when the source supports that mapping. Because ground truth may be incomplete, distinguish `known malicious`, `known benign` and `unknown/unlabeled` rather than forcing every event to binary labels.

- Use labels only for evaluation and optional supervised auxiliary loss; do not include label columns, attack-window flags or post-event investigation annotations in model inputs.
- For window-level evaluation, label a window malicious if the predefined ground-truth rule says it overlaps a mapped malicious event or attack interval. Document the overlap rule and also consider entity/edge-level labels.
- Keep an audit trail from every label to its evidence in the ground-truth report. A label inferred from an IOC match should be marked as an IOC-based label, not as a complete official per-event ground truth.

7. Dataset splitting and leakage prevention

Core rule

Split chronologically before sliding-window generation and before fitting any scaler, encoder or KNN index. A target at time t may use only earlier snapshots; never let overlapping windows cross split boundaries. Keep labels, IOCs and analyst annotations out of model features; document host/process ID reuse, and lock the threshold before evaluating test data.

7.1 Unsupervised / benign-model protocol (recommended primary experiment)

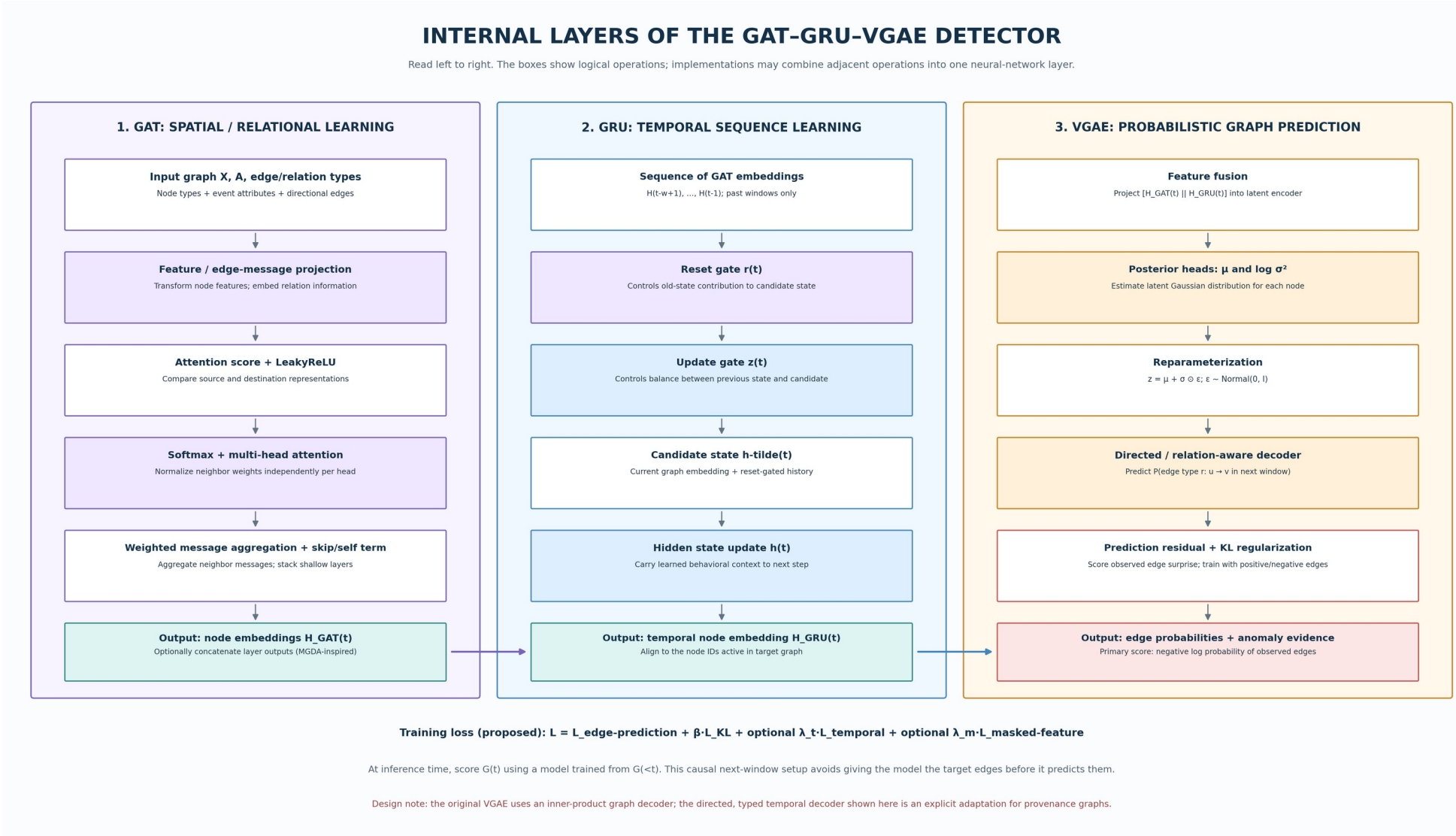
Subset	Content	How it is used
Training	Early or designated benign period(s); no known attack windows.	Fit GAT/GRU/VGAE self-supervised objectives and store benign training embeddings for KNN.
Validation	Later benign windows plus labeled validation attack windows, from a time block not used for training.	Choose model settings, score weights and threshold. Keep the final test untouched.
Test	Held-out later time block and/or held-out attack scenario/chain with surrounding benign behavior.	Report final performance, false-positive rate and detection delay only once.

Do not promise a fixed 70/15/15 split before inspecting the attack and benign timeline. A ratio can be used as an initial planning guide, but temporal boundaries and attack coverage matter more than the ratio. If labels permit, report a separate leave-one-attack-scenario-out experiment for unknown-attack generalization.

7.2 Supervised or hybrid-label protocol (optional)

If you train a supervised detection head, use only training labels for its loss, select all hyperparameters using validation, and test on held-out windows/scenarios. Do not compare a model trained with attack labels against a benign-only model without stating the training difference.

Figure 2. Internal layers of GAT, GRU and VGAE



Original layer diagram. The VGAE decoder shown is a proposed directed/relation-aware temporal link predictor for provenance data, not the original undirected inner-product decoder from the VGAE paper.

8. GAT: internal layers, equations and use

A Graph Attention Network calculates learned attention coefficients over neighboring nodes and uses them to aggregate neighborhood messages. In MGDA, a stacked GAT encoder additionally incorporates an edge embedding into its message so that relation type affects the information passed from source to destination [1,5]. That edge-aware form is particularly appropriate for provenance edges such as READ, WRITE, EXECUTE and CONNECT.

8.1 GAT input

- Node feature matrix X_t: node types and permitted node features for the current snapshot.
- Directed edge index / adjacency A_t: which source and destination entities interacted.
- Relation embedding e_r: identifies edge semantics such as READ or EXECUTE, where the chosen implementation supports edge-aware messages.
- Optional graph metadata: timestamps, host namespace and node/edge IDs for trace-back, kept separate from features unless explicitly selected.

8.2 GAT logical layers

Layer / operation	What happens	Use in APT detection
1. Feature projection	Transform x_i into a learned feature space: h_i = W x_i.	Make categorical and numeric features useful for message passing.
2. Relation-aware message (MGDA-inspired)	m(src,dst) = W_msg^T [h_src || e_edge].	Retain the meaning of the observed interaction, not only the two endpoint identities.
3. Attention score	Compare the destination embedding with the incoming message, then apply LeakyReLU.	Learn which neighboring source/messages matter for the destination representation.
4. Softmax normalization	Normalize scores over the destination node's neighborhood to obtain α(src,dst).	Create relative neighbor weights; these are not maliciousness probabilities.
5. Weighted aggregation	Sum α-weighted neighbor messages and combine with a self/node term.	Build a context-aware representation of a process, file, socket or other entity.
6. Multi-head / stacked layers	Several attention heads run in parallel; stacked layers propagate information across multiple hops.	Represent different relational contexts and short causal neighborhoods.
7. Output embedding	Return H_GAT(t), optionally concatenating intermediate layer representations.	Feed node representations to the temporal model and/or anomaly-scoring branch.

8.3 Core GAT equations

Feature transformation:

```
h_i = W x_i
```

An ordinary attention-style score can be written as:

```
e_ij = LeakyReLU(a^T [W x_i || W x_j])
```

Normalize over the neighbors of i:

```
alpha_ij = exp(e_ij) / sum_{k in N(i)} exp(e_ik)
```

Aggregate neighbor messages:

```
h_i_prime = sigma( sum_{j in N(i)} alpha_ij * W x_j )
```

MGDA's edge-aware version forms messages from the source embedding and an edge embedding, calculates an attention coefficient for the source-to-destination message, normalizes it with softmax, and aggregates the weighted messages into the destination representation [1]. Exact parameterization depends on the selected GAT implementation.

8.4 What GAT can and cannot infer

- Can learn structural context: a process connected to a user, a sensitive file and a remote endpoint may receive a representation different from a process with routine connections.
- Can propagate information over one-hop or multi-hop neighborhoods as layers are stacked.
- Cannot independently prove an APT; its embedding requires a reconstruction, classification or outlier-scoring objective to become a detection signal.
- Attention weights can help inspect model behavior but are not definitive explanations on their own.
- Avoid deep stacks without evaluation; deeper propagation costs more and can blur node representations. Start with two layers and test deeper variants as an ablation.

9. GRU: internal gates, equations and use

A GRU processes the sequence of GAT embeddings for the same entity across recent snapshots (node-level temporal modeling) or a graph-level readout for each snapshot (graph-level temporal modeling). EdgeFence provides a practical reference for fixed-interval temporal graph snapshots, spatial GNN representations, global mean pooling and a single-layer GRU [3]. The recommended MVP uses entity-aligned sequences where feasible; graph-level pooling is an easier baseline but loses some local information.

9.1 GRU inputs

- x_t: the current input embedding at historical time t (e.g., GAT embedding of a given entity in a past snapshot).
- h_(t-1): the previous hidden state, which summarizes earlier inputs in the sequence.
- For a target graph G_t, the model's history ends at G_(t-1). This is necessary for causal next-window prediction.

9.2 GRU logical layers

Gate / state	Equation (one common convention)	Interpretation
Reset gate	r_t = sigmoid(W_r x_t + U_r h_(t-1) + b_r)	Controls how much old hidden-state information enters the candidate-state calculation.
Update gate	z_t = sigmoid(W_z x_t + U_z h_(t-1) + b_z)	Controls the balance between previous state and candidate state.
Candidate state	h_t_tilde = tanh(W_h x_t + U_h (r_t * h_(t-1)) + b_h)	Builds a proposed current representation from the new input and reset-gated history.

Gate / state	Equation (one common convention)	Interpretation
Hidden-state update	h_t = (1-z_t) * h_(t-1) + z_t * h_t_tilde	Combines old state and candidate state; carries the result to the next time step.
Temporal embedding	H_GRU(t) = GRU(H_GAT(t-w), ..., H_GAT(t-1))	Summarizes the ordered recent behavior available before the target window.

Here `*` denotes element-wise multiplication. Some GRU implementations define the update gate in the opposite direction; always follow the convention used by the framework and its documented equations [6].

9.3 Why GRU matters for APTs

A login, a process execution, a credential-file access and a host connection may each occur in benign contexts. The order and temporal spacing can make the combination more concerning. GRU learns temporal dependencies in the sequence; it does not automatically know ATT&CK tactics or automatically generate a prediction error. If temporal prediction error is part of the score, add a prediction head that predicts a future embedding, edge distribution or event category and train it explicitly.

10. VGAE: latent space, decoder and anomaly scoring

A Variational Graph Autoencoder maps nodes to probability distributions in a latent space and decodes those latent representations to model graph structure. The original VGAE formulation uses a GCN encoder and an inner-product decoder over an adjacency matrix [7]. The direction-aware typed edge predictor below is a proposed extension for provenance graphs, not the exact original VGAE architecture.

10.1 VGAE logical layers

Layer	Operation	Purpose
1. Fused feature input	Use current node features plus spatial/temporal context available from past snapshots.	Provides GAT structural context and GRU temporal memory to the graph encoder.
2. Graph encoder	Map each node to a hidden representation using graph context.	Compactly represents relevant graph and node features.
3. Distribution heads	Estimate mean μ_i and log-variance log σ_i^2 for each node.	Defines a latent Gaussian rather than one fixed latent vector.
4. Reparameterization	z_i = μ_i + σ_i ⊙ ε, with ε ~ N(0,I).	Allows stochastic latent sampling while keeping the operation trainable by backpropagation.
5. Relation-aware decoder	Predict P(u --r--> v in the next window | past graph sequence).	Scores directed typed interactions; preferable to undirected links for provenance.
6. Reconstruction / prediction loss	Compare predicted link probabilities with edges observed in the next snapshot; regularize latent distribution with KL divergence.	Teaches the model typical graph transitions and yields an edge-surprise score at inference.

10.2 Latent distribution

```
q(z_i | history) = Normal(mu_i, diag(sigma_i^2))
```

```
z_i = mu_i + sigma_i * epsilon,  epsilon ~ Normal(0, I)
```

The mean μ_i gives the center of the node's learned distribution, and variance σ_i^2 characterizes its spread. In implementation, predict log-variance for numerical convenience and derive σ using the correct stable transformation.

10.3 Original versus proposed decoder

Original VGAE:

```
P(A_ij = 1 | z_i, z_j) = sigmoid(z_i^T z_j)
```

This inner-product decoder is simple but does not by itself preserve direction and multiple relation types. For the proposed provenance-aware hybrid, use an ordered, relation-conditioned decoder, such as:

```
p(u --r--> v at t | history) = sigmoid(MLP([z_u || emb(r) || z_v || time_context]))
```

The ordered concatenation [z_u || emb(r) || z_v] lets the score depend on source-versus-destination order; relation embedding emb(r) distinguishes READ from WRITE or CONNECT. This decoder is a design recommendation that must be implemented and validated, not a claim from the original VGAE paper.

10.4 Why predict the next graph instead of reconstructing the graph the encoder already saw?

If the same target graph is provided to the encoder and then scored by its reconstruction, the encoder may already have access to the edges it is asked to reconstruct. This can make the anomaly score less meaningful. A more defensible temporal detector uses only G_(t-w), …, G_(t-1) to predict G_t, then compares the prediction with the graph that actually arrives at t. That is a temporal adaptation of graph reconstruction, and it follows the no-future-information principle described by EdgeFence for its temporal windows [3].

10.5 VGAE loss

For directed typed link prediction, train on positive edges present in the target snapshot and sampled negative candidate edges. A basic binary link loss is:

```
L_edge = -sum_{e in E+} log p(e | history)
       -sum_{e in E-} log(1 - p(e | history))
```

A KL term regularizes the latent distributions toward the prior, commonly N(0,I):

```
L_KL = -1/2 * sum_i,k [1 + log(sigma_ik^2) - mu_ik^2 - sigma_ik^2]
```

Then:

```
L_VGAE = L_edge + beta * L_KL
```

Graph sparsity requires careful negative sampling or loss weighting. "Missing in telemetry" is not always the same as a known negative event. Use relation-compatible negative edges and inspect the effect of sampling ratios.

11. Proposed hybrid training objective

For the MVP, let GAT encode each historical graph snapshot, let GRU encode the per-entity or graph-level sequence, and let a temporal VGAE-style decoder predict typed edges in the target graph. Use the next-window link-prediction loss plus KL regularization. Add auxiliary losses only after the simple model trains correctly.

```
L_total = L_edge-prediction + beta * L_KL
        + lambda_t * L_temporal (optional)
        + lambda_m * L_masked-feature (optional; MGDA-inspired)
```

Term	When to include	Source alignment
L_edge-prediction	Required for the proposed predictive decoder; positives are target edges and negatives are sampled candidate links.	VGAE link reconstruction [7], plus MGDA sampled structure reconstruction [1].
L_KL	Required for a variational latent model.	Original VGAE objective [7].
L_temporal	Optional; use if the model explicitly predicts next embeddings/events in addition to edges.	Temporal prediction is a proposed extension; EdgeFence supports the use of temporal graph sequences and a GRU [3].
L_masked-feature	Optional advanced pretraining objective: mask selected node features during training and reconstruct them from graph context; never mask at inference.	MGDA multi-view masked reconstruction and latent target prediction [1].

Training loop (conceptual)

```
for epoch in range(max_epochs):
    for history, target_graph in training_sequences:
        # history contains only snapshots before the target time
        gat_seq = [GAT(g.X, g.edge_index, g.edge_type) for g in history]
        temporal_nodes = GRU(align_entity_sequences(gat_seq))
        mu, logvar = VGAE_encoder(fuse(gat_seq[-1], temporal_nodes), history_graph)
        z = reparameterize(mu, logvar)
        edge_prob = directed_typed_decoder(z, candidate_edges_for_target)
        loss = edge_prediction_bce(edge_prob, target_graph.edges) + beta * kl(mu, logvar)
        # Optional: + lambda_t * next-embedding loss
        # Optional: + lambda_m * masked-feature reconstruction loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    evaluate_validation_loss_and_early_stop()
```

This is pseudocode, not a drop-in implementation. Node alignment, missing/new nodes, negative-edge construction, minibatching and the exact graph encoder must be implemented for the chosen data schema.

12. Inference, anomaly score and thresholding

12.1 Causal inference procedure

1. Collect records in the new time window and build the current observed graph G_t.
2. Construct history using only the previous w snapshots G_(t-w), …, G_(t-1).

3. Run GAT on each history graph and GRU on ordered entity embeddings or pooled graph summaries.
4. Use the trained temporal VGAE decoder to estimate probabilities for candidate directed, typed edges in G_t.
5. Compute the edge-surprise score from observed current edges. Optionally also compute a KNN embedding outlier score and a separately trained temporal prediction residual.
6. Normalize or calibrate score components using validation-derived statistics, combine only the components that improve validation performance, and compare with the frozen threshold.
7. If the threshold is crossed, emit an alert with traceable evidence and run bounded investigation tracing.

12.2 Score definitions

Primary predictive edge-surprise score:

```
S_edge(t) = mean over observed edges e in E_t of [-log(p_hat(e | G_<t) + epsilon)]
```

Optional MGDA-inspired embedding-space outlier score:

```
S_knn(v) = normalized mean distance from embedding(v) to its k nearest benign training embeddings
```

Optional temporal residual (only if trained explicitly):

```
S_temp(t) = error(predicted current embedding/features from past, observed current embedding/features)
```

Combine scores only after scaling them on validation data:

```
S_total(t) = w_e * Z(S_edge) + w_k * Z(S_knn) + w_t * Z(S_temp)
```

Weights w_e, w_k and w_t are optional and should be selected on validation, not test. Start with S_edge only, then perform ablations to see whether KNN or temporal residual improves detection. Do not assume that adding more scores makes the system better.

12.3 Threshold selection

- For anomaly detection, a clear starting point is to set a threshold using the held-out benign validation score distribution, such as a percentile chosen to achieve a target false-positive rate.
- EdgeFence reports selecting its threshold to achieve 0.1% validation FPR in its own CIC-IDS/IoT-23 experiments [3]. Treat that as a documented example, not a guaranteed appropriate threshold for DARPA TC.
- If the benchmark requires a target FPR, freeze the threshold that meets it on validation, then report test metrics at that operating point.
- If selecting threshold to maximize F1, describe this separately; it is not the same as a fixed operational FPR threshold.
- Report whether the score is at window level, edge level or entity level. Do not compare metrics across different granularities as if they were identical.

13. Attack scenario reconstruction and alert output

13.1 From anomaly seeds to compact attack graph

MGDA takes detected anomalous process-like nodes as reconstruction seeds, then correlates context and matches TTP rules to produce a semantically enriched scenario graph [1]. Commander demonstrates why

cross-machine tracing and persistence/session awareness can be critical: long-running APT activities may be split across logon sessions or multiple hosts, so a detector should avoid relying only on nearby timestamps [2].

1. Seed selection: select top suspicious entities/edges by calibrated score and minimum evidence requirements.
2. Bounded backward trace: follow causal incoming relationships over a controlled time horizon to find possible parent/trigger context.
3. Bounded forward trace: follow observed downstream dependencies to identify related files, processes, connections and potentially impacted hosts.
4. Cross-host links: follow only observed network/authentication/logon/session relationships; do not infer an identity relationship from a destination IP alone.
5. Rule/context mapping: compare the trace against documented TTP patterns. Mark each technique as `rule_matched`, `candidate` or `unknown` and retain the matching event evidence.
6. Compact graph: merge only semantically equivalent nodes/events while preserving path evidence and timestamps. Keep a pointer from each summary node back to raw event IDs.
7. Threat score: rank alerts using calibrated anomaly evidence plus the number/quality of linked attack indicators and validated TTP matches. Keep a raw model score and a separate rule/incident score.

13.2 Commander ideas that are optional

Commander uses four specialized preliminary detectors for authentication anomalies, persistence, session hijacking and port forwarding, followed by whole-network tracing and TTP-rule ranking [2]. These techniques are highly tied to the log sources and identity/session model available in the paper. For DARPA TC, implement only the cases supported by your chosen performer's records. You can first build general causal tracing and ATT&CK-style rule matching, then add session-specific detectors if the data provides the necessary fields.

13.3 Example JSON output

The following is a synthetic example of the output format—not a detection result from DARPA TC:

```
{
  "alert_id": "APT-2026-00017",
  "window_start": "2026-10-25T10:15:00Z",
  "window_end": "2026-10-25T10:16:00Z",
  "detection_unit": "time_window_and_entity",
  "risk_score": 0.93,
  "threshold": 0.78,
  "decision": "investigate",
  "score_components": {
    "edge_surprise": 0.91,
    "knn_outlier": 0.84,
    "temporal_residual": null
  },
  "entities": [
    {"id": "process:hostA:proc_104", "type": "process"},
    {"id": "file:hostA:file_9", "type": "file"},
    {"id": "host:hostB", "type": "host"}
  ],
  "evidence_event_ids": ["event_001", "event_014", "event_020"],
  "attack_path": [
    "process:hostA:proc_104 --READ--> file:hostA:file_9",
    "process:hostA:proc_104 --CONNECT--> host:hostB"
  ],
  "ttp_matches": [],
  "investigation_status": "new"
}
```

The threshold and scores above are invented for illustration. Real outputs should include model version, dataset/run, score calibration version, trace-back pointers and the exact rule or evidence used to support any attack-stage label. Do not add an ATT&CK technique unless a rule or trained classifier supports the mapping.

13.4 Dashboard views

- Alerts table: timestamp, risk, score components, host, status and analyst disposition.
- Attack timeline: ordered events and time-window boundaries.
- Provenance subgraph: labeled node types, edge types, direction arrows and evidence IDs.
- Model evidence panel: edge probabilities, reconstruction residual, optional KNN score and threshold.
- Performance page: precision, recall, F1, PR-AUC, FPR, detection latency and resource usage.

14. Evaluation plan and ablation experiments

14.1 Metrics

Metric	Formula / meaning	Why report it
Precision	TP / (TP + FP)	How many alerts are true positives among alerted units.
Recall	TP / (TP + FN)	How many labeled malicious units are detected.
F1-score	2 × Precision × Recall / (Precision + Recall)	Balance precision and recall at a stated threshold.
False-positive rate	FP / (FP + TN)	How much benign activity is incorrectly flagged.
PR-AUC / Average Precision	Area under the precision-recall curve / average precision	Useful when malicious windows are rare and class imbalance is high.
Detection delay	Time from first mapped attack event to first alert	Measures whether the system catches a multi-step chain early enough.
Alert burden	Alerts per hour, host or million events	Approximates analyst workload.
Resource cost	Inference time/window, peak RAM, graph size and model size	Indicates feasibility on CPU/low-memory systems.

Use standard TP, FP, TN and FN definitions with one stated detection unit. If overlapping windows are used, report window-level results and, separately, incident/attack-chain-level results. MGDA reports precision, recall, F1 and FPR and discusses efficiency; EdgeFence measures classification and inference-resource metrics [1,3].

14.2 Minimum experiment matrix

Run	Model	Purpose
B0	Static features + logistic regression or small MLP	Sanity baseline; ensure labels/splits work.
B1	GAT embeddings + KNN (MGDA-inspired)	Tests graph representation without temporal memory.

Run	Model	Purpose
B2	GAT + GRU prediction/classification head	Measures benefit of temporal history without VGAE.
B3	GAT + VGAE / edge reconstruction	Measures graph-link anomaly score without GRU history.
B4	Full GAT + GRU + temporal VGAE	Proposed core model.
B5	B4 + KNN score fusion	Tests whether embedding distance adds complementary signal.
B6	B4 + MGDA-style masked-feature auxiliary objective	Advanced self-supervised pretraining ablation.
B7	B4 + causal tracing/TTP rules	Measures interpretability/incident-level value; evaluate rule precision separately.

14.3 Result table template

Model	Precision	Recall	F1	FPR	PR-AUC	Median delay	RAM / inference
B0 — baseline	—	—	—	—	—	—	—
B1 — GAT+KNN	—	—	—	—	—	—	—
B2 — GAT+GRU	—	—	—	—	—	—	—
B3 — GAT+VGAE	—	—	—	—	—	—	—
B4 — full hybrid	—	—	—	—	—	—	—
B5 — full + KNN fusion	—	—	—	—	—	—	—

Do not fill this table with target values or the results reported by the papers. Populate it only with your own completed experiments. Use at least three random seeds if the compute budget allows; report mean and standard deviation or confidence intervals where feasible.

15. API/dashboard and repository layout

15.1 Suggested repository structure

```
apt-detection/
  README.md
  configs/
    e3_cadets_mvp.yaml
    model_gat_gru_vgae.yaml
  data/
    raw/              # not committed to Git
    decoded/          # streamed JSONL, not committed if large
```

```
  interim/          # normalized Parquet
  processed/        # graph/window manifests
  labels/           # ground-truth-to-event mapping
  splits/           # fixed train/validation/test manifests
src/
  ingestion/        # release-specific decoder wrapper
  preprocessing/    # normalize, validate, compress
  graph/            # node/edge mapping, window builder
  labels/           # IOC/ground-truth mapping
  models/
    gat_encoder.py
    gru_temporal.py
    temporal_vgae.py
    fusion.py
  training/         # objectives, loops, checkpoints
  scoring/          # residuals, KNN, threshold
  investigation/    # tracing, TTP rules, attack graph output
  api/              # FastAPI routes
  dashboard/        # Streamlit or Grafana integration
tests/
  test_parser.py
  test_graph_direction.py
  test_no_future_leakage.py
  test_labels_not_features.py
  test_score_threshold.py
artifacts/
  model/            # checkpoints, scalers, feature schema
  reports/          # metrics and plots
notebooks/          # exploratory work only; production logic stays in src/
```

15.2 Suggested API endpoints

Endpoint	Purpose	Example response
GET /health	Check service status.	{"status":"ok"}
POST /score-window	Score a graph-window sequence already normalized and built.	score components + threshold decision
GET /alerts	List stored alerts with pagination/filtering.	Alert rows with status and scores
GET /alerts/{id}	Return one alert and evidence IDs.	Alert JSON, paths, linked events
GET /alerts/{id}/graph	Return bounded investigation graph for visualization.	Nodes and directed edges with labels
POST /alerts/{id}/disposition	Record analyst disposition; do not silently retrain from it.	Updated status and audit trail

15.3 Inference service flow

```
new raw chunk
  -> parser / schema validation
  -> normalized events + entity dictionary
  -> update graph snapshot and temporal history buffer
  -> run GAT/GRU/VGAE inference
  -> calibrate score and apply frozen threshold
  -> if flagged: persist alert + event IDs + bounded trace graph
  -> API/dashboard renders evidence for analyst review
```

16. Implementation roadmap and acceptance criteria

Phase	Tasks	Acceptance criteria / artifact
P0 — Scope and setup	Pick E3 performer, record dataset version, configure environment/repository.	README contains dataset choice, source links, environment versions, scope and reproducible config.
P1 — Parser	Decode a small sample using release-matched tools.	Parser test passes; records and timestamps are inspectable; raw data is not modified.
P2 — Normalization	Create canonical event schema and quality report.	Missing fields, unknown types and duplicates are counted and documented.
P3 — Graph/windows	Build directed typed snapshots and temporal sequences.	Unit tests verify edge direction, IDs, timestamps and sequence boundaries.
P4 — Ground truth/splits	Map labels separately; define train/validation/test boundaries.	Split manifests fixed; target window contains no future data; labels never enter feature vectors.
P5 — Baselines	Run simple features/KNN and a static GAT experiment.	First reproducible metric table; errors and runtime logged.
P6 — GRU	Add the temporal sequence model and explicit temporal objective if using residual.	B2 result compared on identical split; temporal ablation and leakage tests pass.
P7 — VGAE	Add mean/log-variance heads, decoder, edge loss and KL loss.	Training converges; edge probabilities and residuals are sanity-checked on toy graphs.
P8 — Score calibration	Compare edge residual, KNN and any temporal residual.	Threshold selected using validation only and stored with model version.
P9 — Investigation graph	Implement bounded tracing and rules supported by data.	Alert includes raw evidence pointers and never emits unsupported ATT&CK labels.
P10 — Evaluation	Run held-out test and ablations.	Final report contains metrics, runtime/resource use, false-positive review and limitations.
P11 — Demo	Wrap scoring with API and dashboard.	A test graph yields an alert JSON and readable investigation graph.
P12 — Optional research extension	Add masked feature reconstruction, adversarial evaluation or FL only after baseline.	Each extension has a separate ablation, documented threat model and reproducible result.

17. Common failure modes and mitigations

Failure mode	Why it happens	Mitigation
Parser schema mismatch	E3/E5 or different TA1 performers have different release/schema assumptions.	Pin the release, use its README/schema/consumer, verify record types before full processing.

Failure mode	Why it happens	Mitigation
Memory crash	Decoded events and graph adjacency can be far larger than compressed input.	Stream/chunk; save Parquet; graph by time window; sample neighbors/subgraphs; avoid dense N×N adjacency matrices.
High false-positive rate	Training data is contaminated or benign rare events are unfamiliar.	Use quality-checked benign training data, target-FPR threshold, per-alert evidence, false-positive review and robust split.
Poor recall	Sparse labels, incomplete telemetry, missing cross-host relations or too-large windows.	Inspect ground-truth mappings, test window-size sweep, preserve event coverage and validate attack path links.
Leaky test results	Random windows overlap in time or preprocessing uses test statistics.	Split chronologically before windowing; fit scalers/KNN on training only; keep test labels out of threshold tuning.
VGAE learns to copy the target graph	The encoder sees target edges used for scoring.	Use past-only inputs to predict next-window edges, or mask target edges during training and evaluation with a defined protocol.
Undirected decoder loses relation semantics	Standard inner-product VGAE is not directed/typed by default.	Use an ordered relation-aware decoder; document it as a proposed extension; add typed-edge reconstruction tests.
GRU input does not align across snapshots	Node sets change from window to window.	Use stable entity IDs and explicit padding/masks, or start with graph-level pooling as a baseline.
ATT&CK overclaim	An anomaly score is not the same as a technique classifier.	Only map technique when a rule/classifier has supporting event evidence; otherwise output “unknown/candidate”.
Adding all methods reduces clarity	Masked AE, VGAE, KNN, FL, CKKS and adversarial training complicate one another.	Build core GAT+GRU+VGAE first; add one extension at a time and use ablations.

18. Final deliverables and viva explanation

Required project deliverables

- Dataset manifest and release/schema note; the dataset itself may be too large to include in the repository.
- Parser and normalized event schema, data quality report and graph-window construction code.
- Ground-truth mapping and leakage-safe split manifests.
- Trained model artifacts for GAT, GRU and VGAE; configuration and version metadata.
- Baseline/ablation results including precision, recall, F1, FPR, PR-AUC, delay and resource usage.
- Architecture diagram, model-layer diagram, algorithm pseudocode and final project report.
- Alert JSON schema, example alerts with traceable evidence, attack graph visualization and dashboard/API demonstration.
- Limitations section distinguishing anomaly detection from definitive attribution and future attack prediction.

Short viva explanation

“First, the system decodes DARPA Transparent Computing audit records and converts them into directed provenance graphs. GAT learns which relationships between users, processes, files and hosts are useful. GRU

reads the graph embeddings in chronological order so the detector can model behavior across time windows. A temporal VGAE-style decoder uses the learned latent representations to predict the next window's typed graph edges. When an observed edge is unlikely under the model, it receives a higher anomaly score. A validation-calibrated threshold identifies windows for investigation. Finally, bounded provenance tracing and supported TTP rules create an evidence-backed alert and compact attack scenario graph. We compare the full model with simpler baselines to verify which components actually improve detection."

Terminology to use accurately

Call the first version an APT anomaly-detection / investigation system. Use "prediction" specifically for the next-window edge/behavior forecast. Claim future attack prediction only if you define a future target/horizon and evaluate it separately.

19. References and paper-to-design traceability

[1] M. Cui, Z. Jiang, S. Li, C. Ma, K. Zhang, P. Yang, and H. Feng, "MGDA: A provenance graph-based framework for threat detection and attack scenario reconstruction," Computer Networks, vol. 274, article 111806, 2026. DOI: https://doi.org/10.1016/j.comnet.2025.111806. (Uploaded PDF: Provenance_Graph_Threat_Detection_Attack_Reconstruction.)

[2] Q. Liu, K. Bao, and V. Hagenmeyer, "Commander: A robust cross-machine multi-phase Advanced Persistent Threat detector via provenance analytics," Journal of Information Security and Applications, vol. 91, article 104057, 2025. DOI: https://doi.org/10.1016/j.jisa.2025.104057. (Uploaded PDF: Robust_Cross_Machine_Multi_Phase_APT_Detection_via_Provenance_Analytics.)

[3] O. Isaac, B. Appiah, D. Commey, K. Owusu-Agyemang, M. Asante, and B. H. Acquah, "EdgeFence: Federated Temporal Graph Neural Networks for Lightweight, Adversarial Malware Detection in Distributed Edge Networks," International Journal of Information Security, vol. 25, article 88, 2026. DOI: https://doi.org/10.1007/s10207-026-01237-x. (Uploaded PDF: Federated_Temporal_GNN_Malware_Detection.)

[4] DARPA, "Transparent Computing Engagement 3 Data Release," official repository README-E3 and Transparent Computing program overview. https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md and https://www.darpa.mil/research/programs/transparent-computing.

[5] P. Veličković et al., "Graph Attention Networks," ICLR 2018; arXiv:1710.10903. https://arxiv.org/abs/1710.10903.

[6] K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP 2014. https://aclanthology.org/D14-1179/.

[7] T. N. Kipf and M. Welling, "Variational Graph Auto-Encoders," arXiv:1611.07308, 2016. https://arxiv.org/abs/1611.07308.

[8] PyTorch Geometric, "GATConv" and "VGAE" API documentation. https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.conv.GATConv.html and https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.models.VGAE.html.

Paper-to-design page guide

Source	Useful part in uploaded PDF	Used for
MGDA [1]	Provenance graph construction and compression (Sec. 4.1); masked GAT representation learning, edge reconstruction, KNN anomaly detector and TTP reconstruction (Secs. 4.2–4.4); DARPA E3 and metrics (Sec. 5).	Data schema and graph encoding, optional masked pretraining, KNN outlier scoring, investigation graph.
Commander [2]	System overview / Fig. 6 and three stages; preliminary detectors; whole-network tracing; attack graph ranking (Secs. 6.1–6.3).	Cross-host causal tracing, session/persistence awareness where data supports it, TTP-based graph ranking.
EdgeFence [3]	Temporal snapshots and window definitions (Sec. 3.2.1); spatial GNN + pooling + GRU model (Sec. 3.2.2); inference/thresholding, FedAvg and adversarial training (Secs. 3.2.3–3.2.5); experimental settings (Sec. 4).	Temporal window construction, GRU role, causal labeling, validation threshold and optional FL/robustness extensions.
DARPA E3 README [4]	E3 release outline, ground-truth report, Java consumer, schema files and known-good topic list.	Dataset selection, parser setup and release matching.